

Code-Layout, Readability & Best Practices

HDI Prediction System — Project Development Phase

Project Structure

The HDI Prediction System follows a clean, modular Flask project layout that separates data, model, templates, and application logic for readability and easy maintenance.

Folder Layout

- `app.py` — main Flask application handling routes and predictions.
- `model/HumDevIndex.ipynb` — notebook used for model training and evaluation.
- `model/hdi_model.pkl` — serialized trained Linear Regression model.
- `templates/` — HTML files for input form and result display (dark theme).
- `static/` — CSS and any static assets for styling.
- `hdi.csv` — source dataset used for training.

Code Readability Practices

- Descriptive variable and function names (e.g. `life_expectancy`, `predict_hdi`) instead of single letters.
- Consistent indentation and PEP-8 style formatting throughout the Flask app.
- Form field names in HTML kept identical to model feature names to avoid mapping errors.
- Comments added at key logic points such as model loading and prediction handling.
- Routes kept short and single-purpose — one route for the form, one for the prediction result.

Best Practices Followed

- Model loaded once at application startup rather than on every request, for efficiency.
- Input validation on the form to prevent empty or invalid indicator values.
- Separation of concerns: ML logic isolated from presentation (HTML/CSS) logic.
- Version control with Git — regular, descriptive commits pushed to GitHub.
- Environment-specific issues (e.g. Windows PATH) documented for team reference.

Maintainability

Keeping the model, templates, and application logic in clearly separated folders makes it straightforward for any team member to update the dataset, retrain the model, or restyle the interface without affecting other parts of the system.